



Interface binária de aplicação

Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

Introdução

- *Application Binary Interface* (ABI)
  - Define as convenções para interface em nível de binário entre aplicações, bibliotecas ou sistemas operacionais

Introdução

- *Application Binary Interface* (ABI)
  - Define as convenções para interface em nível de binário entre aplicações, bibliotecas ou sistemas operacionais
  - É dependente do *hardware* e a portabilidade depende da padronização da ABI pelos sistemas

Introdução

- *Application Binary Interface* (ABI)
  - Define as convenções para interface em nível de binário entre aplicações, bibliotecas ou sistemas operacionais
  - É dependente do *hardware* e a portabilidade depende da padronização da ABI pelos sistemas

Não é *Application Programming Interface* (API)

Introdução

- ABI $\times$ API
  - A principal diferença para a ABI reside no fato da API ser utilizada em nível de linguagem de programação

Introdução

- ABI $\times$ API
  - A principal diferença para a ABI reside no fato da API ser utilizada em nível de linguagem de programação
  - Define como o programador pode fazer uso das funções disponíveis de bibliotecas ou SO

Introdução

- ABI $\times$ API
  - A principal diferença para a ABI reside no fato da API ser utilizada em nível de linguagem de programação
  - Define como o programador pode fazer uso das funções disponíveis de bibliotecas ou SO
  - É convertida pelo compilador em formato binário utilizando a ABI definida pelo sistema

Introdução

- Principais funções da ABI
  - Alinhamento e tamanho de tipos de dados

Introdução

- Principais funções da ABI
  - Alinhamento e tamanho de tipos de dados
  - Sequência de chamada de funções

Introdução

- Principais funções da ABI
  - Alinhamento e tamanho de tipos de dados
  - Sequência de chamada de funções
  - Interface com o sistema operacional

Representação dos dados

- Define como os diferentes tipos de dados são armazenados e representados
  - Tipos escalares de ANSI C
  - Agregados e uniões

Representação dos dados

- Tipos escalares na ABI de 32 bits

Tipo	C	Tamanho (*sizeof*)	Alinhamento (*bytes*)	*Intel386*
Inteiro	**char** **signed char**	1	1	**signed byte**
	unsigned char	1	1	**unsigned byte**
	short **signed short**	2	2	**signed** **halfword**
	unsigned short	2	2	**unsigned** **halfword**
	int **signed int** **long signed long** **enum**	4	4	**signed word**
	unsigned int **unsigned long**	4	4	**unsigned word**

Representação dos dados

- Tipos escalares na ABI de 32 bits

Tipo	C	Tamanho (*sizeof*)	Alinhamento (*bytes*)	*Intel386*
Pon-teiro	* (*)()	4	4	**unsigned word**
Ponto flutu-ante	**float**	4	4	**single-precision (IEEE)**
	double	8	4	**double-precision (IEEE)**
	long double	12	4	**extended-precision (IEEE)**

Representação dos dados

- Tipos escalares na ABI de 64 bits

Tipo	C	Tamanho (*sizeof*)	Alinhamento (*bytes*)	*AMD64*
Inteiro	**bool**	1	1	**boolean**
	char **signed char**	1	1	**signed byte**
	unsigned char	1	1	**unsigned byte**
	short **signed short**	2	2	**signed twobyte**
	unsigned short	2	2	**unsigned twobyte**
	int **signed int** **enum**	4	4	**signed fourbyte**
	unsigned int	4	4	**unsigned fourbyte**

Representação dos dados

- Tipos escalares na ABI de 64 bits

Tipo	C	Tamanho (*sizeof*)	Alinhamento (*bytes*)	*AMD64*
Inteiro	**long** **signed long** **long long** **signed long ong**	8	8	**signed eightbyte**
	unsigned long **unsigned long long**	8	8	**unsigned eightbyte**
	__int128 **signed __int128**	16	16	**signed sixteenbyte**
	unsigned __int128	16	16	**unsigned sixteenbyte**

Representação dos dados

- Tipos escalares na ABI de 64 bits

Tipo	C	Tamanho (*sizeof*)	Alinha-mento (*bytes*)	*AMD64*
Pon-teiro	* (*)()	8	8	**unsigned eightbyte**
Ponto flutu-ante	**float**	4	4	**single (IEEE-754)**
	double	8	8	**double (IEEE-754)**
	long double	16	16	**80-bit extended (IEEE-754)**
	__float128	16	16	**128-bit extended (IEEE-754)**

Representação dos dados

- Tipos escalares na ABI de 64 bits

Tipo	C	Tamanho (*sizeof*)	Alinha-mento (*bytes*)	*AMD64*
Ponto flutuante decimal	**_Decimal32**	4	4	**32bit BID (IEEE-754R)**
	_Decimal64	8	8	**64bit BID (IEEE-754R)**
	_Decimal128	16	16	**128bit BID (IEEE-754R)**
Enpaco-tado	**_m64**	8	8	**MMX/3D Now!**
	_m128	16	16	**SSE/SSE-2**
	_m256	32	32	**AVX**

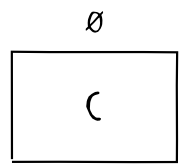
Representação dos dados

- Agregados e uniões na ABI
  - Os agregados (estruturas e vetores) são alinhados pelo membro mais estritamente alinhado
  - Para cada membro é atribuído o menor deslocamento disponível mantendo o alinhamento
  - O tamanho da estrutura pode ser aumentada para a tornar múltipla do alinhamento
  - No preenchimento (*padding*), o conteúdo utilizado nesta operação é indefinido

Representação dos dados

- Agregados e uniões na ABI de 32 bits

```
1 struct {
2     char c;
3 };
```

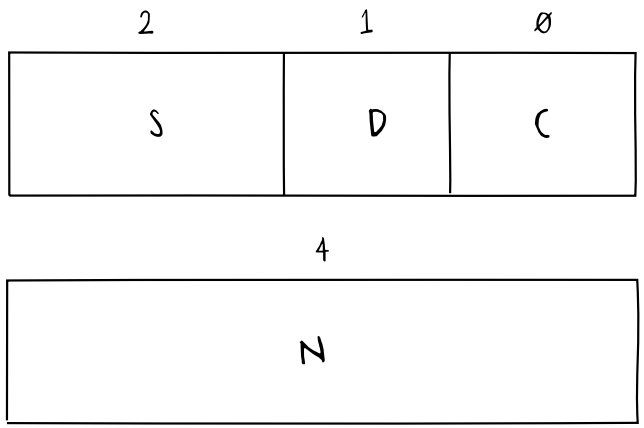


Alinhado por byte (1 byte)

Representação dos dados

- Agregados e uniões na ABI de 32 bits
  - Sem preenchimento (*padding*)

```
1  struct {
2      char c;
3      char d;
4      short s;
5      long n;
6  };
```

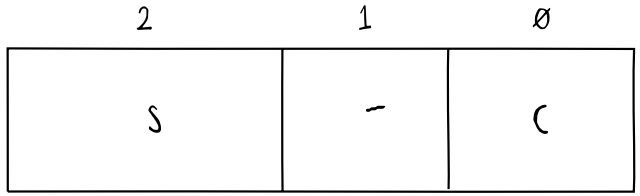


Alinhado por palavra (8 bytes)

Representação dos dados

- Agregados e uniões na ABI de 32 bits
  - Preenchimento interno

```
1  struct {
2      char c;
3      short s;
4  };
```

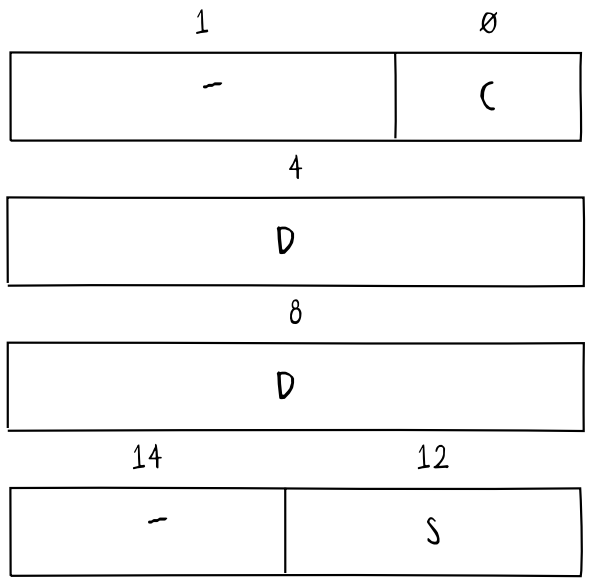


Alinhado por meia palavra (4 bytes)

Representação dos dados

- Agregados e uniões na ABI de 32 bits
  - Preenchimento externo e interno

```
1  struct {
2      char c;
3      double d;
4      short s;
5  };
```

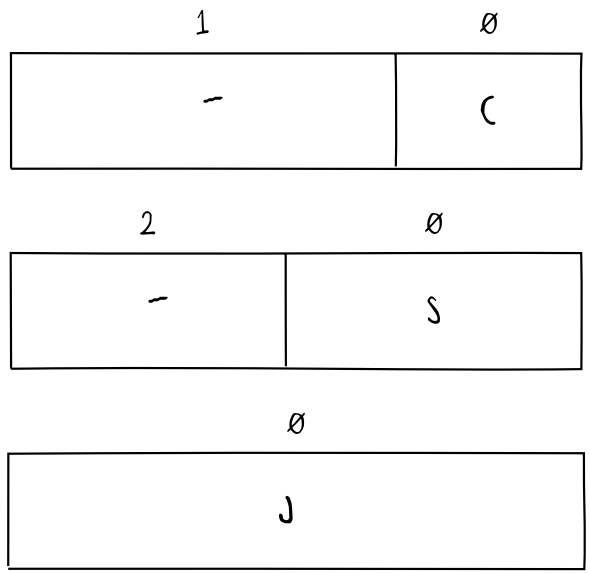


Alinhado por palavra (16 bytes)

Representação dos dados

- Agregados e uniões na ABI de 32 bits
  - Alocação sobreposta

```
1  union {
2      char c;
3      short s;
4      int j;
5  };
```



Alinhado por palavra (4 bytes)

Exercício

- Aprofunde o estudo das especificações da ABI
  - Verifique a implementação dos campos de bit (*bit-fields*), considerando o alinhamento e o preenchimento dos dados
  - Realize experimentos para verificar a organização dos dados para tipos agregados na ABI de 64 bits (AMD64), incluindo tipos de dados de 8 e 16 bytes
  - Entenda os tipos de dados de ponto flutuante decimal e empacotado, além de suas aplicações

Chamadas de função

- Definição de padrões para chamadas de função
  - Organização da pilha
  - Utilização dos registradores
  - Passagem de parâmetros e retorno de valores

Chamadas de função

- Organização da pilha (32 bits)
  - Alinhamento de palavra (4 bytes)
  - Registradores **ebp** (apontador do quadro da pilha) e **esp** (apontador do topo da pilha)

Endereço superior

↓

Endereço inferior

Posição	Conteúdo	Quadro
4n+8 (**ebp**)	Argumento n	Anterior
⋮	⋮	
8 (**ebp**)	Argumento 0	
4 (**ebp**)	Endereço de retorno	Atual
0 (**ebp**)	**ebp** anterior	
-4 (**ebp**)	Não especificado	
0 (**esp**)		

Chamadas de função

- Organização da pilha (64 bits)
  - Alinhamento de 16 bytes nos argumentos de entrada
  - Registradores **rbp** (apontador do quadro da pilha) e **rsp** (apontador do topo da pilha)

Endereço superior

↓

Endereço inferior

Posição	Conteúdo	Quadro
8n+16 (**rbp**)	Argumento n	Anterior
⋮	⋮	
16 (**rbp**)	Argumento 0	
8 (**rbp**)	Endereço de retorno	Atual
0 (**rbp**)	**rbp** anterior	
-8 (**rbp**)	Não especificado	
0 (**rsp**)		

Chamadas de função

- Utilização dos registradores (32 bits)

Tipo	Nome	Utilização	Preservado
Geral	**eax**	Valor de retorno	✗
	edx	Dividendo	✗
	ecx	Contador	✗
	ebx	Variável local	✓
	ebp	Quadro da pilha	✓
	esi	Variável local	✓
	edi	Variável local	✓
	esp	Topo da pilha	✓
Ponto flutuante	**st**(0)	Topo da pilha	✗
	⋮	⋮	⋮
	st(7)	Base da pilha	✗

Chamadas de função

- Utilização dos registradores (64 bits)

Nome	Utilização	Preservado
rax	Valor de retorno 1	✗
rbx	Propósito geral	✓
rcx	Argumento 4	✗
rdx	Argumento 3 / Valor de retorno 2	✗
rsp	Topo da pilha	✓
rbp	Quadro da pilha	✓
rsi	Argumento 2	✗
rdi	Argumento 1	✗
r8	Argumento 5	✗
r9	Argumento 6	✗
r10	Temporário	✗
r11	Temporário	✗
r12-r14	Propósito geral	✓
r15	Propósito geral / Apontador base	✓

Chamadas de função

- Utilização dos registradores (64 bits)

Nome	Utilização	Preservado
xmm0-xmm1	Passagem e retorno de argumentos	✗
xmm2-xmm7	Passagem de argumentos	✗
xmm8-xmm15	Temporário	✗
mmx0-mmx7	Temporário	✗
st0-st1	Temporário (**long double**)	✗
st2-st7	Temporário	✗
fs	Reservado	✗
mxcsr	Controle e status do SSE2	Parcial
x87 SW	Status do x87	✗
x87 CW	Controle do x87	✓

Chamadas de função

- Exemplo de passagem e retorno de parâmetros

```
1  #include <stdio.h>
2  #include <stdint.h>
3  // Função exemplo
4  uint64_t f(uint8_t a, char* b, uint32_t c, int64_t d,
       char e, uint16_t f, void* g) {
5      return -2;
6  }
7  // Função principal
8  int main() {
9      f(1, "2", 271, -314, '5', 6, NULL);
10     return 0;
11 }
```

Chamadas de função

- ABI de 32 bits (Intel386)

```
f:
    push ebp
    mov ebp, esp
    ...
    mov eax, 0xFFFFFFFE
    mov edx, 0xFFFFFFFF
    mov esp, ebp
    pop ebp
    ret
main:
    ...
    push 0x00000000
    push 0x0006
    push 0x35
    push 0xFFFFFFFF
    push 0xFFFFFEC6
    push 0x0000010F
    push [b]
    push 0x01
    call f
    ...
```

Chamadas de função

- ABI de 32 bits (Intel386)

```
f:
    push ebp
    mov ebp, esp
    ...
    mov eax, 0xFFFFFFFE
    mov edx, 0xFFFFFFFF
    mov esp, ebp
    pop ebp
    ret
main:
    ...
    push 0x00000000
    push 0x0006
    push 0x35
    push 0xFFFFFFFF
    push 0xFFFFFEC6
    push 0x0000010F
    push [b]
    push 0x01
    call f
    ...
```

Chamadas de função

- ABI de 32 bits (Intel386)

```
f:
    push ebp
    mov ebp, esp
    ...
    mov eax, 0xFFFFFFFE
    mov edx, 0xFFFFFFFF
    mov esp, ebp
    pop ebp
    ret
main:
    ...
    push 0x00000000
    push 0x0006
    push 0x35
    push 0xFFFFFFFF
    push 0xFFFFFEC6
    push 0x0000010F
    push [b]
    push 0x01
    call f
    ...
```

Chamadas de função

- ABI de 32 bits (Intel386)

```
f:
    push ebp
    mov ebp, esp
    ...
    mov eax, 0xFFFFFFFE
    mov edx, 0xFFFFFFFF
    mov esp, ebp
    pop ebp
    ret
main:
    ...
    push 0x00000000
    push 0x0006
    push 0x35
    push 0xFFFFFFFF
    push 0xFFFFFEC6
    push 0x0000010F
    push [b]
    push 0x01
    call f
    ...
```

Chamadas de função

- ABI de 32 bits (Intel386)

```
f:
    push ebp
    mov ebp, esp
    ...
    mov eax, 0xFFFFFFFE
    mov edx, 0xFFFFFFFF
    mov esp, ebp
    pop ebp
    ret
main:
    ...
    push 0x00000000
    push 0x0006
    push 0x35
    push 0xFFFFFFFF
    push 0xFFFFFEC6
    push 0x0000010F
    push [b]
    push 0x01
    call f
    ...
```

Chamadas de função

- ABI de 32 bits (Intel386)

```
f:
    push ebp
    mov ebp, esp
    ...
    mov eax, 0xFFFFFFFE
    mov edx, 0xFFFFFFFF
    mov esp, ebp
    pop ebp
    ret
main:
    ...
    push 0x00000000
    push 0x0006
    push 0x35
    push 0xFFFFFFFF
    push 0xFFFFFEC6
    push 0x0000010F
    push [b]
    push 0x01
    call f
    ...
```

Chamadas de função

- ABI de 64 bits (AMD64)

```
f:
    push rbp
    mov rbp, rsp
    ...
    mov rax, 0xFFFFFFFFFFFFFFFE
    mov rsp, rbp
    pop rbp
    ret
main:
    ...
    push 0x00000000
    mov r9, 0x0006
    mov r8, 0x35
    mov rcx, 0xFFFFFFFFFFFFFEC6
    mov edx, 0x0000010F
    mov esi, [b]
    mov edi, 0x01
    call f
    ...
```

Chamadas de função

- ABI de 64 bits (AMD64)

```
f:
    push rbp
    mov rbp, rsp
    ...
    mov rax, 0xFFFFFFFFFFFFFFFE
    mov rsp, rbp
    pop rbp
    ret
main:
    ...
    push 0x00000000
    mov r9, 0x0006
    mov r8, 0x35
    mov rcx, 0xFFFFFFFFFFFFEC6
    mov edx, 0x0000010F
    mov esi, [b]
    mov edi, 0x01
    call f
    ...
```

Chamadas de função

- ABI de 64 bits (AMD64)

```
f:
    push rbp
    mov rbp, rsp
    ...
    mov rax, 0xFFFFFFFFFFFFFFFE
    mov rsp, rbp
    pop rbp
    ret
main:
    ...
    push 0x00000000
    mov r9, 0x0006
    mov r8, 0x35
    mov rcx, 0xFFFFFFFFFFFFFEC6
    mov edx, 0x0000010F
    mov esi, [b]
    mov edi, 0x01
    call f
    ...
```

Chamadas de função

- ABI de 64 bits (AMD64)

```
f:
    push rbp
    mov rbp, rsp
    ...
    mov rax, 0xFFFFFFFFFFFFFFFE
    mov rsp, rbp
    pop rbp
    ret
main:
    ...
    push 0x00000000
    mov r9, 0x0006
    mov r8, 0x35
    mov rcx, 0xFFFFFFFFFFFFFEC6
    mov edx, 0x0000010F
    mov esi, [b]
    mov edi, 0x01
    call f
    ...
```

Chamadas de função

- ABI de 64 bits (AMD64)

```
f:
    push rbp
    mov rbp, rsp
    ...
    mov rax, 0xFFFFFFFFFFFFFFFE
    mov rsp, rbp
    pop rbp
    ret
main:
    ...
    push 0x00000000
    mov r9, 0x0006
    mov r8, 0x35
    mov rcx, 0xFFFFFFFFFFFFFEC6
    mov edx, 0x0000010F
    mov esi, [b]
    mov edi, 0x01
    call f
    ...
```

Chamadas de função

- ABI de 64 bits (AMD64)

```
f:
    push rbp
    mov rbp, rsp
    ...
    mov rax, 0xFFFFFFFFFFFFFFFE
    mov rsp, rbp
    pop rbp
    ret
main:
    ...
    push 0x00000000
    mov r9, 0x0006
    mov r8, 0x35
    mov rcx, 0xFFFFFFFFFFFFEC6
    mov edx, 0x0000010F
    mov esi, [b]
    mov edi, 0x01
    call f
    ...
```

Exercício

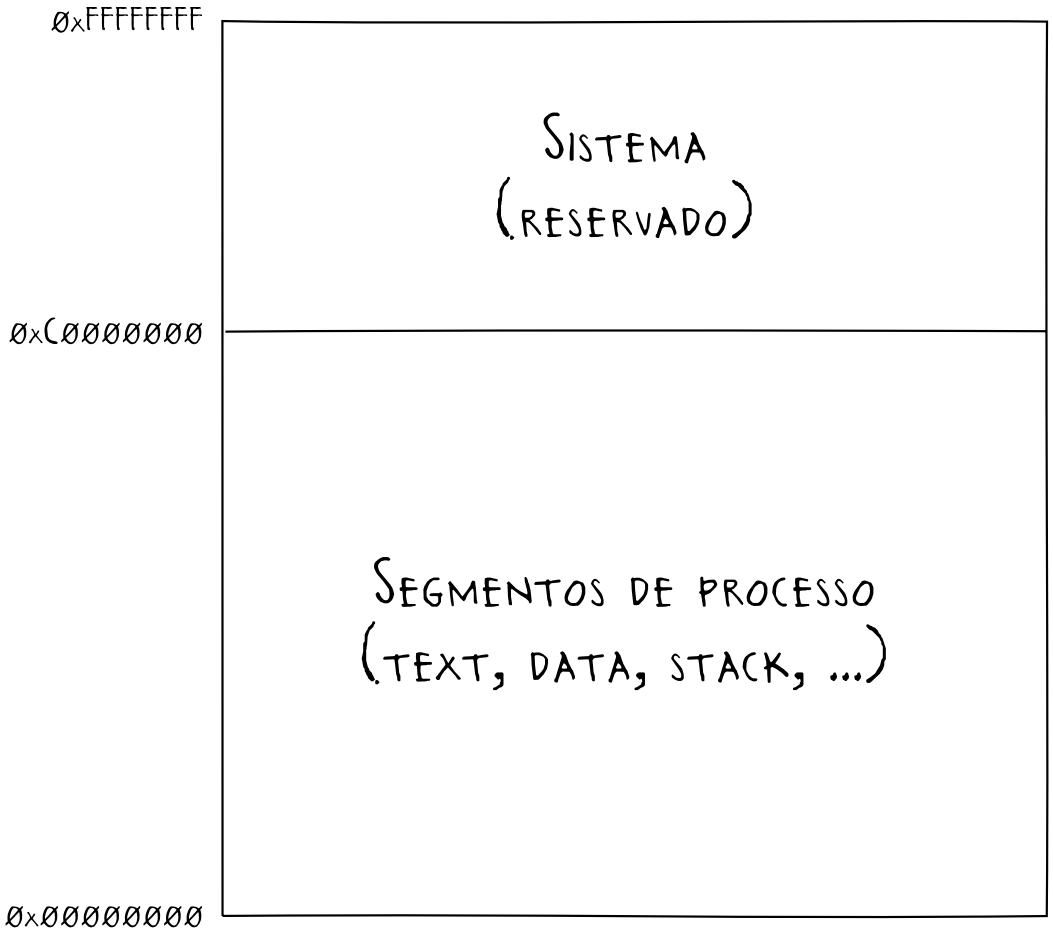
- Estude as convenções de chamadas de função nos manuais da Intel386 e AMD64
  - Desmonte o código binário de exemplos para aprofundar o entendimento da ABI
  - Faça experimentos com parâmetros de ponto flutuante com precisão simples e dupla
  - Compare as especificações com EABI e Microsoft x64, em termos de compatibilidade e desempenho

Interface com SO

- Definições de padrões para interface com SO
  - Espaço de endereços virtuais e tamanho de página
  - Códigos para exceções e sinais
  - Inicialização da pilha do processo

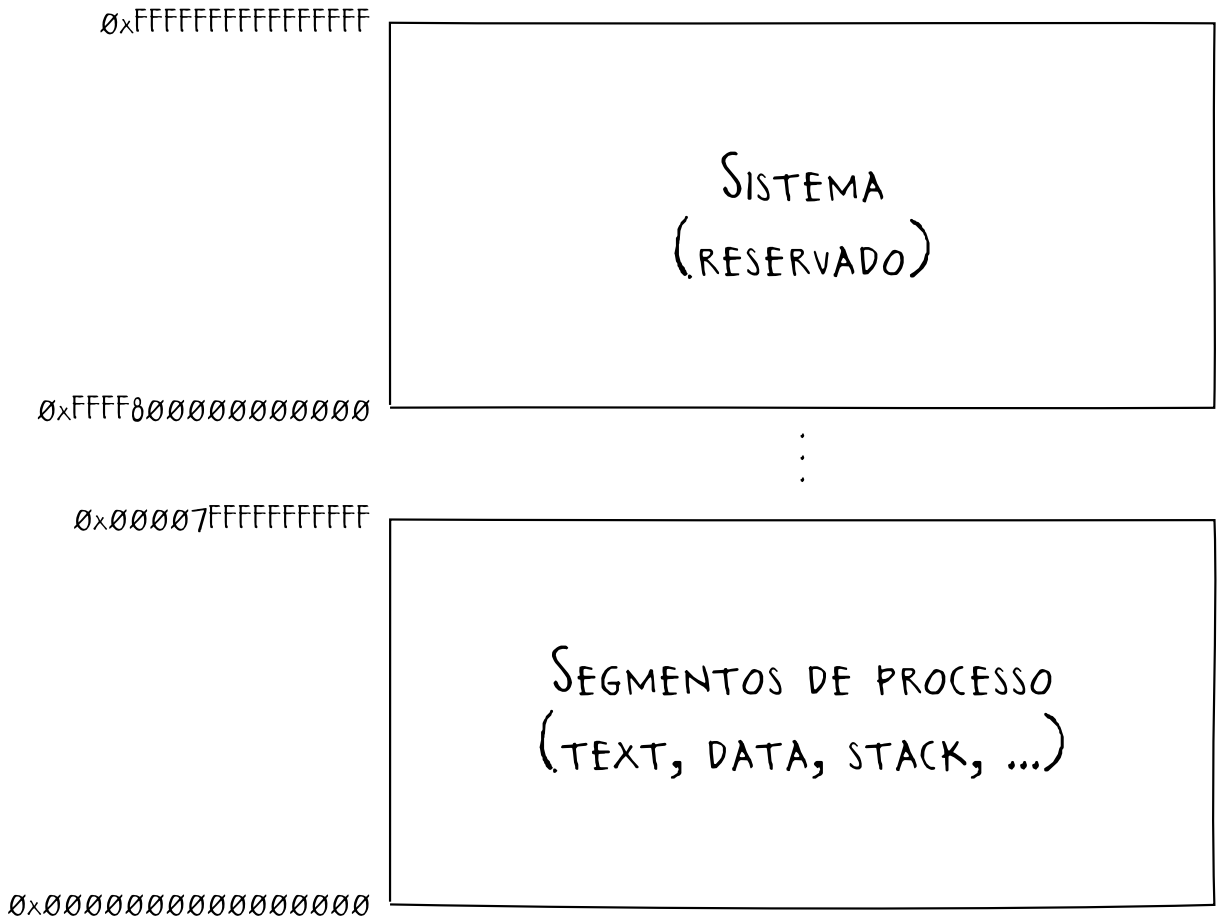
Interface com SO

- Mapa de memória virtual (32 bits)



Interface com SO

- Mapa de memória virtual (64 bits)
  - Faixa de endereçamento de 48 bits



Interface com SO

- Tamanho da página
  - Precisa ser um valor potência de 2 entre 4 KiB e 64 KiB
  - Este parâmetro depende do processador, da unidade de gerenciamento de memória (MMU) e das configurações do sistema operacional

```
1 #include <stdio.h>
2 #include <unistd.h>
3 int main() {
4     printf("Tamanho␣da␣página␣=␣%lu␣bytes\n",
          sysconf(_SC_PAGESIZE));
5     return 0;
6 }
```

Interface com SO

- Tamanho da página
  - Precisa ser um valor potência de 2 entre 4 KiB e 64 KiB
  - Este parâmetro depende do processador, da unidade de gerenciamento de memória (MMU) e das configurações do sistema operacional

```c
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("Tamanho da página = %lu bytes\n",
        sysconf(_SC_PAGESIZE));
    return 0;
}
```

Tamanho da página = 4096 bytes

Interface com SO

- Códigos para exceções e sinais

Número	Descrição	Sinal
0	erro de divisão	SIGFPE
1	captura/falta de passo simples	SIGTRAP
2	interrupção não mascarável	-
3	captura de ponto de parada	SIGTRAP
4	captura de *overflow*	SIGSEGV
5	(reservado)	
6	código de operação inválido	SIGILL
7	falta de coprocessor	SIGFPE
8	aborto de dupla falta	-

Interface com SO

- Códigos para exceções e sinais

Número	Descrição	Sinal
9	aborto de sobrecarga de coprocessador	SIGSEGV
10	falta inválida de TSS	-
11	falta de segmento	-
12	falta de exceção de pilha	SIGSEGV
13	falta/aborto de proteção geral	SIGSEGV
14	falta de página	SIGSEGV
15	(reservado)	
16	falta de erro de coprocessador	SIGFPE
outro	(não especificado)	SIGILL

Interface com SO

- Exceções para ponto flutuante

Código	Causa
FPE_FLTDIV	Divisão por zero
FPE_FLTOVF	*Overflow*
FPE_FLTUND	*Underflow*
FPE_FLTRES	Resultado inexato
FPE_FLTINV	Operação inválida

Interface com SO

- Inicialização da pilha do processo (64 bits)

Propósito	Endereço inicial	Tamanho
Não especificado	Endereços superiores	
Bloco de informações		Variado
Não especificado		
Vetor de entrada nulo auxiliar		8 bytes / cada
Entradas de vetor auxiliar		16 bytes / cada
0		8 bytes
Ponteiro de ambiente		8 bytes / cada
0	**rsp**+8+8*__argc__	8 bytes
Ponteiro de argumentos (**argv**)	***rsp****+8*	**argc** * 8 bytes
Contador de argumentos (**argc**)	**rsp**	8 bytes
Não definido	Endereços inferiores	

Interface com SO

- Inicialização da pilha do processo (64 bits)

```c
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
int main(int argc, char* argv[]) {
    uint32_t i;
    printf("argc = %u\n", argc);
    for(i = 0; i < argc; i++)
        printf("argv[%u] = %s\n", i, argv[i]);
    return 0;
}
```

```
$ gcc -Wall -g exemplo.c -o exemplo.elf
$ gdb exemplo.elf
...
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from exemplo.elf...
(gdb) b _start
Breakpoint 1 at 0x1040
(gdb)
```

Interface com SO

- Inicialização da pilha do processo (64 bits)

```c
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
int main(int argc, char* argv[]) {
    uint32_t i;
    printf("argc = %u\n", argc);
    for(i = 0; i < argc; i++)
        printf("argv[%u] = %s\n", i, argv[i]);
    return 0;
}
```

```
Type "apropos word" to search for commands related to "word"...
Reading symbols from exemplo.elf...
(gdb) b _start
Breakpoint 1 at 0x1040
(gdb) r abc def
Starting program: exemplo.elf abc def

Breakpoint 1, 0x0000555555555040 in _start ()
(gdb)
```

Interface com SO

- Inicialização da pilha do processo (64 bits)

```c
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
int main(int argc, char* argv[]) {
    uint32_t i;
    printf("argc = %u\n", argc);
    for(i = 0; i < argc; i++)
        printf("argv[%u] = %s\n", i, argv[i]);
    return 0;
}
```

```
Breakpoint 1 at 0x1040
(gdb) r abc def
Starting program: exemplo.elf abc def

Breakpoint 1, 0x0000555555555040 in _start ()
(gdb) p/x $rsp
$1 = 0x7fffffffe0b0
(gdb) p/x $rbp
$2 = 0x0
```

Interface com SO

- Inicialização da pilha do processo (64 bits)

```c
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
int main(int argc, char* argv[]) {
    uint32_t i;
    printf("argc = %u\n", argc);
    for(i = 0; i < argc; i++)
        printf("argv[%u] = %s\n", i, argv[i]);
    return 0;
}
```

```
(gdb) p/x $rbp
$2 = 0x0
(gdb) x/40b $rsp
0x7fffffffe0b0: 0x03 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7fffffffe0b8: 0xa2 0xe4 0xff 0xff 0xff 0x7f 0x00 0x00
0x7fffffffe0c0: 0xc2 0xe4 0xff 0xff 0xff 0x7f 0x00 0x00
0x7fffffffe0c8: 0xc6 0xe4 0xff 0xff 0xff 0x7f 0x00 0x00
0x7fffffffe0d0: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
(gdb)
```

Interface com SO

- Inicialização da pilha do processo (64 bits)

```
7f...e0b0:  03  00  00  00  00  00  00  00
7f...e0b8:  a2  e4  ff  ff  ff  7f  00  00
7f...e0c0:  c2  e4  ff  ff  ff  7f  00  00
7f...e0c8:  c6  e4  ff  ff  ff  7f  00  00
7f...e0d0:  00  00  00  00  00  00  00  00
              :   :   :   :   :   :   :   :
```

```
(gdb) b main
Breakpoint 2 at 0x555555555148: file exemplo.c, line 6.
(gdb) c
Continuing.
Breakpoint 2, main (argc=3, argv=0x7fffffffe0b8) at exemplo.c:6
6               printf("argc = %u\n", argc);
```

Interface com SO

- Inicialização da pilha do processo (64 bits)

```
7f...e0b0:  03  00  00  00  00  00  00  00  argc
7f...e0b8:  a2  e4  ff  ff  ff  7f  00  00
7f...e0c0:  c2  e4  ff  ff  ff  7f  00  00
7f...e0c8:  c6  e4  ff  ff  ff  7f  00  00
7f...e0d0:  00  00  00  00  00  00  00  00
          :   :   :   :   :   :   :   :   :
```

```
(gdb) b main
Breakpoint 2 at 0x555555555148: file exemplo.c, line 6.
(gdb) c
Continuing.
Breakpoint 2, main (argc=3, argv=0x7fffffffe0b8) at exemplo.c:6
6               printf("argc = %u\n", argc);
(gdb) p/x argc
$3 = 0x3
```

Interface com SO

- Inicialização da pilha do processo (64 bits)

```
7f...e0b0:  03  00  00  00  00  00  00  00  argc
7f...e0b8:  a2  e4  ff  ff  ff  7f  00  00  argv
7f...e0c0:  c2  e4  ff  ff  ff  7f  00  00
7f...e0c8:  c6  e4  ff  ff  ff  7f  00  00
7f...e0d0:  00  00  00  00  00  00  00  00
            :   :   :   :   :   :   :   :   :
```

```
(gdb) b main
Breakpoint 2 at 0x555555555148: file exemplo.c, line 6.
(gdb) c
Continuing.
Breakpoint 2, main (argc=3, argv=0x7fffffffe0b8) at exemplo.c:6
6               printf("argc = %u\n", argc);
(gdb) p/x argc
$3 = 0x3
(gdb) p/x argv
$4 = 0x7fffffffe0b8
```

Interface com SO

- Inicialização da pilha do processo (64 bits)

```
7f...e0b0:  03  00  00  00  00  00  00  00    argc
7f...e0b8:  a2  e4  ff  ff  ff  7f  00  00  argv[0]
7f...e0c0:  c2  e4  ff  ff  ff  7f  00  00
7f...e0c8:  c6  e4  ff  ff  ff  7f  00  00
7f...e0d0:  00  00  00  00  00  00  00  00
          :   :   :   :   :   :   :   :   :
```

```
(gdb) b main
Breakpoint 2 at 0x555555555148: file exemplo.c, line 6.
(gdb) c
Continuing.
Breakpoint 2, main (argc=3, argv=0x7fffffffe0b8) at exemplo.c:6
6               printf("argc = %u\n", argc);
(gdb) p/x argc
$3 = 0x3
(gdb) p/x argv
$4 = 0x7fffffffe0b8
(gdb) p/x argv[0]
$5 = 0x7fffffffe4a2
```

Interface com SO

- Inicialização da pilha do processo (64 bits)

```
7f...e0b0:  03  00  00  00  00  00  00  00    argc
7f...e0b8:  a2  e4  ff  ff  ff  7f  00  00  argv[0]
7f...e0c0:  c2  e4  ff  ff  ff  7f  00  00  argv[1]
7f...e0c8:  c6  e4  ff  ff  ff  7f  00  00
7f...e0d0:  00  00  00  00  00  00  00  00
            :   :   :   :   :   :   :   :   :
```

```
(gdb) c
Continuing.
Breakpoint 2, main (argc=3, argv=0x7fffffffe0b8) at exemplo.c:6
6               printf("argc = %u\n", argc);
(gdb) p/x argc
$3 = 0x3
(gdb) p/x argv
$4 = 0x7fffffffe0b8
(gdb) p/x argv[0]
$5 = 0x7fffffffe4a2
(gdb) p/x argv[1]
$6 = 0x7fffffffe4c2
```

Interface com SO

- Inicialização da pilha do processo (64 bits)

```
7f...e0b0:  03  00  00  00  00  00  00  00    argc
7f...e0b8:  a2  e4  ff  ff  ff  7f  00  00  argv[0]
7f...e0c0:  c2  e4  ff  ff  ff  7f  00  00  argv[1]
7f...e0c8:  c6  e4  ff  ff  ff  7f  00  00  argv[2]
7f...e0d0:  00  00  00  00  00  00  00  00
          :   :   :   :   :   :   :   :   :
```

```
Breakpoint 2, main (argc=3, argv=0x7fffffffe0b8) at exemplo.c:6
6               printf("argc = %u\n", argc);
(gdb) p/x argc
$3 = 0x3
(gdb) p/x argv
$4 = 0x7fffffffe0b8
(gdb) p/x argv[0]
$5 = 0x7fffffffe4a2
(gdb) p/x argv[1]
$6 = 0x7fffffffe4c2
(gdb) p/x argv[2]
$7 = 0x7fffffffe4c6
```

Interface com SO

- Inicialização da pilha do processo (64 bits)

```
7f...e0b0:  03  00  00  00  00  00  00  00    argc
7f...e0b8:  a2  e4  ff  ff  ff  7f  00  00  argv[0]
7f...e0c0:  c2  e4  ff  ff  ff  7f  00  00  argv[1]
7f...e0c8:  c6  e4  ff  ff  ff  7f  00  00  argv[2]
7f...e0d0:  00  00  00  00  00  00  00  00
          :   :   :   :   :   :   :   :   :
```

```
(gdb) p/x argv
$4 = 0x7fffffffe0b8
(gdb) p/x argv[0]
$5 = 0x7fffffffe4a2
(gdb) p/x argv[1]
$6 = 0x7fffffffe4c2
(gdb) p/x argv[2]
$7 = 0x7fffffffe4c6
(gdb) x/4b argv[2]
0x7fffffffe4c6: 0x64    0x65    0x66    0x00
(gdb) x/4b argv[1]
0x7fffffffe4c2: 0x61    0x62    0x63    0x00
```

Exercício

- Realize experimentos para aprofundar o entendimento sobre as funcionalidades desempenhadas pela pilha
  - Verifique as diferenças entre alocação estática e dinâmica no escopo local de uma função
  - Compare implementações de funções iterativas e recursivas a partir do código de montagem gerado
  - Entenda a zona vermelha (*red zone*) da pilha

Exercício

- Implemente em linguagem de montagem (AMD64) um conversor de imagens em escala de cinza em RLE e pixels com 4 bits para representações textuais (ASCII)
  - Formato do arquivo de entrada

$p_0 \ \cdots \ p_{m-1}$
$\#Imagens\ (n)$
$\#m_o\ x_0\ y_0\ b_0\ \cdots\ x_{m_0-1}\ y_{m_0-1}\ b_{m_0-1}$
$\vdots$
$\#m_{n-1}\ x_0\ y_0\ b_0\ \cdots\ x_{m_{n-1}-1}\ y_{m_{n-1}-1}\ b_{m_{n-1}-1}$

```
1 2E␣27␣22␣3A␣2D␣3D␣2B␣2A␣7C␣4C␣5A␣4D␣2A␣23␣25␣40
2 1
3 31␣01␣02␣20␣05␣03␣1F␣01␣01␣20␣08␣00␣10␣00␣02␣1F␣04␣00␣
      10␣01␣00␣20␣00␣00␣1F␣04␣01␣10␣09␣00␣4F␣04␣04␣10␣04␣
      03␣10␣0B␣03␣5F␣05␣00␣3F␣06␣01␣30␣05␣04␣1F␣0A␣01␣20␣
      03␣00␣1F␣09␣01␣1F␣08␣02␣10␣09␣02␣4F␣09␣04␣3F␣05␣02␣
      3F␣03␣01␣1F␣06␣04␣30␣01␣03␣20␣04␣02␣10␣05␣01␣1F␣03␣
      03␣1F␣03␣02␣1F␣06␣03␣50
```

Exercício

- Implemente em linguagem de montagem (AMD64) um conversor de imagens em escala de cinza em RLE e pixels com 4 bits para representações textuais (ASCII)
  - Formato do arquivo de saída

    [*Taxa de compressão*]

    $$\begin{matrix} j_{0,0} & \cdots & j_{0,w_i-1} \\ \vdots & \ddots & \vdots \\ j_{h_i-1,0} & \cdots & j_{h_i-1,w_i-1} \end{matrix}$$

```
1 [155%]
2 @..@.@@@.@@@
3 @..@.@...@..
4 @..@.@@@.@@@
5 @..@.@.....@
6 @@@@.@...@@@
```